

Preempção em Single-Core: Uma Ficção Perigosa

Uma Crítica aos RTOS Preemptivos e a Proposta de um Modelo Cooperativo Determinístico

Marcos Roberto Braga

Instituto Informal de Educação, Ciência e Tecnologia

noelmrb_tec@yahoo.com

September 12, 2026

Abstract

Este trabalho apresenta uma crítica conceitual ao uso indiscriminado de RTOS preemptivos em microcontroladores single-core. Argumentamos que a preempção, tal como implementada em sistemas como o FreeRTOS, constitui uma ficção arquitetural: em single-core não há paralelismo real, apenas serialização rápida com troca de contexto. Esta ficção introduz problemas reais (race conditions, deadlocks, inversão de prioridade, jitter de timing) que não existiriam em um modelo cooperativo determinístico. Propomos o CDC-P (Concurrency Deterministic Control with Preemption), um kernel cooperativo que implementa preempção real por reordenação de fila de prioridades, sem interrupção brusca de tarefas. Resultados empíricos em cinco arquiteturas (PIC16, PIC18, AVR, RISC-V, ARM Cortex-M) demonstram que o modelo proposto é mais determinístico, mais eficiente em recursos e mais seguro que o modelo preemptivo clássico, especialmente em sistemas de tempo real crítico (hard real-time).

Palavras-chave: RTOS, preempção, determinismo, sistemas de tempo real, cooperativo, microcontroladores.

1 Introdução

A indústria de sistemas embarcados adotou, nas últimas duas décadas, uma crença generalizada: *sistemas com múltiplas tarefas exigem RTOS preemptivo*. Esta crença é raramente questionada, e sua adoção é frequentemente justificada por argumentos de autoridade (“*todo mundo usa*”) ou por falácias circulares (“*sistemas complexos exigem complexidade*”).

Este trabalho questiona essa crença em três frentes:

1. **Frente técnica:** demonstramos que a preempção em single-core não é paralelismo real, mas time-slicing, e que esta distinção é sistematicamente ignorada pela literatura e pela indústria.
2. **Frente conceitual:** argumentamos que a preempção introduz problemas (race conditions, deadlocks, jitter) que não existiriam em um modelo cooperativo determinístico, e que estes problemas exigem soluções complexas (mutexes, herança de prioridade, seções críticas) que a própria preempção tornou necessárias.
3. **Frente prática:** propomos o CDC-P (Concurrency Deterministic Control with Preemption), um kernel cooperativo que implementa preempção real por reordenação de fila, e validamos empiricamente em cinco arquiteturas de microcontroladores.

1.1 A Analogia do Pronto-Socorro

Considere um pronto-socorro com um único médico e cinco pacientes. Os pacientes chegam com diferentes urgências. A prática padrão da indústria (RTOS preemptivo) corresponderia a:

“O médico começa a operar o paciente A. No meio da cirurgia, o paciente B chega e o médico interrompe a operação de A, salva o estado (tórax aberto), e vai atender B. No meio da cirurgia de B, o paciente C chega. O médico interrompe B, salva o estado, e vai atender C. E assim por diante. Nenhum paciente sai bem.”

A prática correta (CDC-P) corresponderia a:

*“O médico avalia todos os pacientes. Decide qual é o mais crítico. Opera o mais crítico **completo**. Depois opera o próximo. E assim por diante. Todos os pacientes saem bem.”*

A analogia expõe o absurdo da preempção brusca: **ninguém operaria dois corações ao mesmo tempo**. Mas é exatamente isso que o FreeRTOS faz quando preempta uma tarefa no meio da execução.

1.2 Contribuições

Este trabalho apresenta as seguintes contribuições:

- Uma crítica epistemológica à preempção em single-core;
- Uma análise técnica dos problemas introduzidos pela preempção;
- Uma proposta de modelo cooperativo determinístico com preempção por reordenação de fila;
- Validação empírica em cinco arquiteturas de MCU;
- Dados comparativos com o modelo preemptivo clássico.

2 Fundamentação Teórica

2.1 Liu & Layland (1973)

O paper fundacional de Liu e Layland [1] estabeleceu as bases teóricas do escalonamento de tempo real preemptivo. Suas premissas são:

- Deadline igual ao período;
- Tarefas independentes;
- Ausência de recursos compartilhados;
- Ausência de race conditions;
- Ausência de overhead.

Na prática, **nenhuma destas premissas se verifica**. Tarefas compartilham recursos, race conditions ocorrem, overhead existe. Portanto, a garantia matemática de Liu e Layland é teórica, não real.

2.2 Dijkstra (1968, 1972)

Dijkstra [2,3] estabeleceu dois princípios fundamentais:

1. “*Go To Statement Considered Harmful*” (1968): o programador deve conseguir raciocinar sobre o estado do programa em qualquer ponto.
2. “*Simplicity is prerequisite for reliability*” (1972): a simplicidade é pré-requisito para a confiabilidade.

Argumentamos que a preempção viola o primeiro princípio (é um *goto* disfarçado: salva o PC em um lugar, restaura de outro, troca o fluxo de controle por baixo do código) e ignora o segundo (introduz complexidade que não existiria em um modelo cooperativo).

2.3 Rate Monotonic Scheduling e suas Limitações

O Rate Monotonic Scheduling (RMS) [1] estabelece que a utilização máxima do processador é dada por:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1) \quad (1)$$

onde C_i é o tempo de execução e T_i é o período da tarefa i . Para $n \rightarrow \infty$, $U \rightarrow \ln 2 \approx 0.693$.

No entanto, esta fórmula **ignora o overhead de troca de contexto**. Em um sistema com muitas tarefas e deadlines curtos, o overhead pode consumir 10–30% da CPU, invalidando a garantia teórica.

3 A Contradição da Preempção em Single-Core

3.1 Definição de Tempo Real

Tempo real **não significa “rápido”**. Significa: *o sistema responde dentro de um prazo previsível*. Existem dois tipos:

- **Hard Real-Time:** perder um deadline é uma falha catastrófica (airbag, marca-passo);
- **Soft Real-Time:** perder um deadline é uma degradação aceitável (streaming, jogos).

Para **hard real-time**, o que importa é o **determinismo**, não a velocidade.

3.2 O Argumento Padrão da Indústria

A indústria afirma:

“Use preempção quando há exigências rígidas de tempo real.”

Este argumento é **contraditório**:

1. Preempção introduz **jitter** (latência imprevisível);
2. Jitter **quebra** garantias matemáticas;
3. Portanto, preempção é **inadequada** para hard real-time.

O modelo cooperativo, por outro lado, é **determinístico**:

- Sem jitter de escalonamento;
- Sem race conditions;
- Sem deadlocks;
- Sem inversão de prioridade.

Para hard real-time, cooperativo é melhor.

3.3 A Ficção do Paralelismo

Em single-core, não há paralelismo real. O que existe é:

- **Troca de contexto:** salvar registradores, carregar novos;
- **Fatia de tempo:** tarefa A roda 10ms, B roda 10ms;
- **Interrupção:** timer interrompe A, kernel decide rodar B.

Isso **não é paralelismo**. É **faz de conta**. É uma ficção útil para vender ferramentas, mas perigosa em produção.

4 Problemas Introduzidos pela Preempção

4.1 Race Conditions

Como tarefas não são atômicas, qualquer acesso a dados compartilhados pode corromper. Isso exige **mutexes**, que introduzem:

- **Inversão de prioridade:** tarefa de baixa prioridade segura mutex;
- **Deadlocks:** duas tarefas esperando recursos mutuamente;
- **Starvation:** tarefa nunca consegue o recurso.

4.2 Inversão de Prioridade

O caso clássico é o **Mars Pathfinder** (1997), onde uma tarefa de baixa prioridade bloqueou uma de alta prioridade, causando resets repetidos. A solução exigiu *priority inheritance* no VxWorks.

4.3 Jitter de Timing

Cada troca de contexto introduz latência de 5–15 μ s (Cortex-M). Em 1000 trocas/s, isso consome 10ms/s. Mais grave: o **jitter** acumulado faz o relógio do sistema derrapar.

4.4 Relógios Impedidos

Sistemas com FreeRTOS frequentemente apresentam **congelamento** e **salto** em relógios de segundo. Isso ocorre porque a tarefa do relógio depende de `vTaskDelay`, que:

- Arredonda para múltiplos do tick (jitter de 1–10ms);
- Depende de escalonamento (latência de 1–10ms);
- Pode ser bloqueada por mutex (latência variável).

RTC externo não resolve, porque a leitura do RTC também está sujeita aos mesmos problemas.

5 O Modelo CDC-P

5.1 Visão Geral

O CDC-P (Concurrency Deterministic Control with Preemption) é um kernel cooperativo que implementa preempção real por **reordenação de fila**, sem interrupção brusca de tarefas. Suas características principais são:

- **Tarefas atômicas:** cada tarefa é uma função que sempre retorna;
- **Escalonamento por matemática modular:** $(\text{tick} - \text{timer_task}) \geq \text{periodo}$;
- **Prioridade tridimensional:** espacial (ordem), temporal (período), situacional (URG-S);
- **7 camadas de proteção:** auto-regulagem, bloqueio por deadline, Xerife, Síndico, fail-safe, CDC-P, URG-S;
- **Sem troca de contexto:** stack compartilhada.

5.2 Preempção por Reordenação

O CDC-P implementa dois mecanismos de preempção:

5.2.1 URG-S (Urgência Situacional)

Evento externo seta uma flag. O despachador verifica a flag **antes de tudo**. A tarefa urgente executa **na próxima varredura**. Latência: tempo de varredura (25–112 μ s). Depois, a flag é limpa automaticamente.

5.2.2 CDC-P (Aceleração Persistente)

Aplicação chama `CDC_EnablePreempt`. O período da tarefa é reduzido a 1 tick. A tarefa executa em todas as iterações até `CDC_DisablePreempt`.

5.3 Analogia com Pronto-Socorro

O URG-S corresponde a: *“Paciente crítico chega; médico termina o atendimento atual e atende o crítico em seguida.”*

O CDC-P corresponde a: *“Hospital entra em modo epidemia; todos os recursos se dedicam aos casos críticos enquanto a epidemia durar.”*

Em ambos os casos, preempção real ocorre sem interrupção brusca.

6 Validação Empírica

6.1 Plataformas Testadas

O CDC-P foi implementado em cinco arquiteturas:

6.2 Testes de Stress

Sob 1000 eventos/s (URG-S + CDC-P), o sistema permaneceu estável:

Table 1: Plataformas de validação

MCU	Arquitetura	Clock	RAM	Flash
PIC16F628A	8 bits	4 MHz	224 B	2 KB
PIC18F6722	8 bits	8 MHz	4 KB	128 KB
AT90S8515	AVR 8 bits	8 MHz	512 B	8 KB
CH32V003F4P6	RISC-V	48 MHz	2 KB	16 KB
STM32F103	ARM Cortex-M3	72 MHz	20 KB	64 KB

Table 2: Resultados do teste de stress (1000 Hz)

Métrica	FreeRTOS (est.)	CDC-P (medido)
Custo por sinalização	5–15 μ s	0.02 μ s
Carga CPU a 1000 Hz	0.5–1.5%	0.002%
Troca de contexto	Obrigatória	Inexistente
Latência URG-S	1–2 ticks	Varredura (μ s)
Latência CDC-P	1–2 ticks	1 tick
Stack por tarefa	200–500 B	0 B
Overhead do kernel	100–200 μ s	25 μ s
Determinismo	Degrada	Constante
Precisão do relógio	Deriva	Exata

6.3 Recursos em PIC16F628A

7 Discussão

7.1 Determinismo vs. Velocidade

Os resultados demonstram que o CDC-P é **mais determinístico** que o FreeRTOS, mantendo **precisão de relógio** mesmo sob carga. Isso confirma a tese de que, para hard real-time, o determinismo é mais importante que a velocidade bruta.

7.2 Recursos vs. Complexidade

O CDC-P usa uma fração dos recursos do FreeRTOS, mantendo as mesmas capacidades (preempção, prioridade, semáforos). Isso confirma a tese de que a complexidade do FreeRTOS é autorreferente: cada mecanismo existe para resolver um problema criado pelo mecanismo anterior.

7.3 Portabilidade

O CDC-P foi portado para cinco arquiteturas sem alteração no kernel. Isso confirma a tese de que o modelo cooperativo é naturalmente portátil, ao contrário do modelo preemptivo (que exige port específico para cada arquitetura).

7.4 A Ficção da Preempção

A preempção em single-core é uma **ficção útil**. Ela é útil para vender ferramentas, mas perigosa em produção. Em multi-core, a preempção é **real**: dois núcleos, dois fluxos, paralelismo verdadeiro.

A indústria deveria distinguir os dois casos.

Table 3: Uso de recursos em PIC16F628A

Recurso	Usado	Livre	Total
ROM	1360 words (66%)	688 words (34%)	2048 words
RAM	80 bytes (36%)	144 bytes (64%)	224 bytes
Stack	4 níveis (50%)	4 níveis (50%)	8 níveis

8 Conclusão

Este trabalho apresentou uma crítica conceitual ao uso indiscriminado de RTOS preemptivos em microcontroladores single-core. Argumentamos que:

1. A preempção em single-core é uma ficção: não há paralelismo real;
2. Esta ficção introduz problemas (race, deadlock, jitter) que não existiriam em modelo cooperativo;
3. O modelo cooperativo é mais determinístico e mais adequado para hard real-time;
4. O CDC-P demonstra que preempção real pode ser implementada por reordenação de fila, sem interrupção brusca;
5. Resultados empíricos em cinco arquiteturas confirmam as teses.

A indústria precisa rever seus pressupostos. Nem todo sistema precisa de FreeRTOS. Em muitos casos, o modelo cooperativo determinístico é mais adequado, mais eficiente e mais seguro.

“Preempção real só é possível em multi-core. O que os RTOS preemptivos fazem em single-core é um faz de conta perigoso.”

Agradecimentos

O autor agradece a Pedro Henrique Cerqueira Braga pela colaboração no desenvolvimento e testes do CDC-P.

References

- [1] LIU, C. L.; LAYLAND, J. W. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, v. 20, n. 1, p. 46–61, 1973.
- [2] DIJKSTRA, E. W. Go To Statement Considered Harmful. *Communications of the ACM*, v. 11, n. 3, p. 147–148, 1968.
- [3] DIJKSTRA, E. W. The Humble Programmer. *Communications of the ACM*, v. 15, n. 10, p. 859–866, 1972.
- [4] LABROSSE, J. J. *μC/OS-III: The Real-Time Kernel*. Micrium, 2010.
- [5] FreeRTOS Documentation. <https://www.freertos.org/>. Acesso em: 2026.
- [6] BARR, M. *Programming Embedded Systems in C and C++*. O’Reilly, 1999.
- [7] JONES, M. What Really Happened on Mars? *NASA JPL*, 1997.

- [8] HYMEL, S. When to Use an RTOS. *Digi-Key Blog*, 2023.
- [9] HARVIE, L. The Uncomfortable Truth About RTOS in Embedded Systems. *LinkedIn*, 2024.
- [10] ASTIER, J. Y. Multiple FSMs vs RTOS. *LinkedIn*, 2024.